



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

13. Parallelismo

Linguaggi e Compilatori [I215-011]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-262]
Anno accademico 2022/2023

Prof. Andrea Marongiu
andrea.marongiu@unimore.it

Copyright note

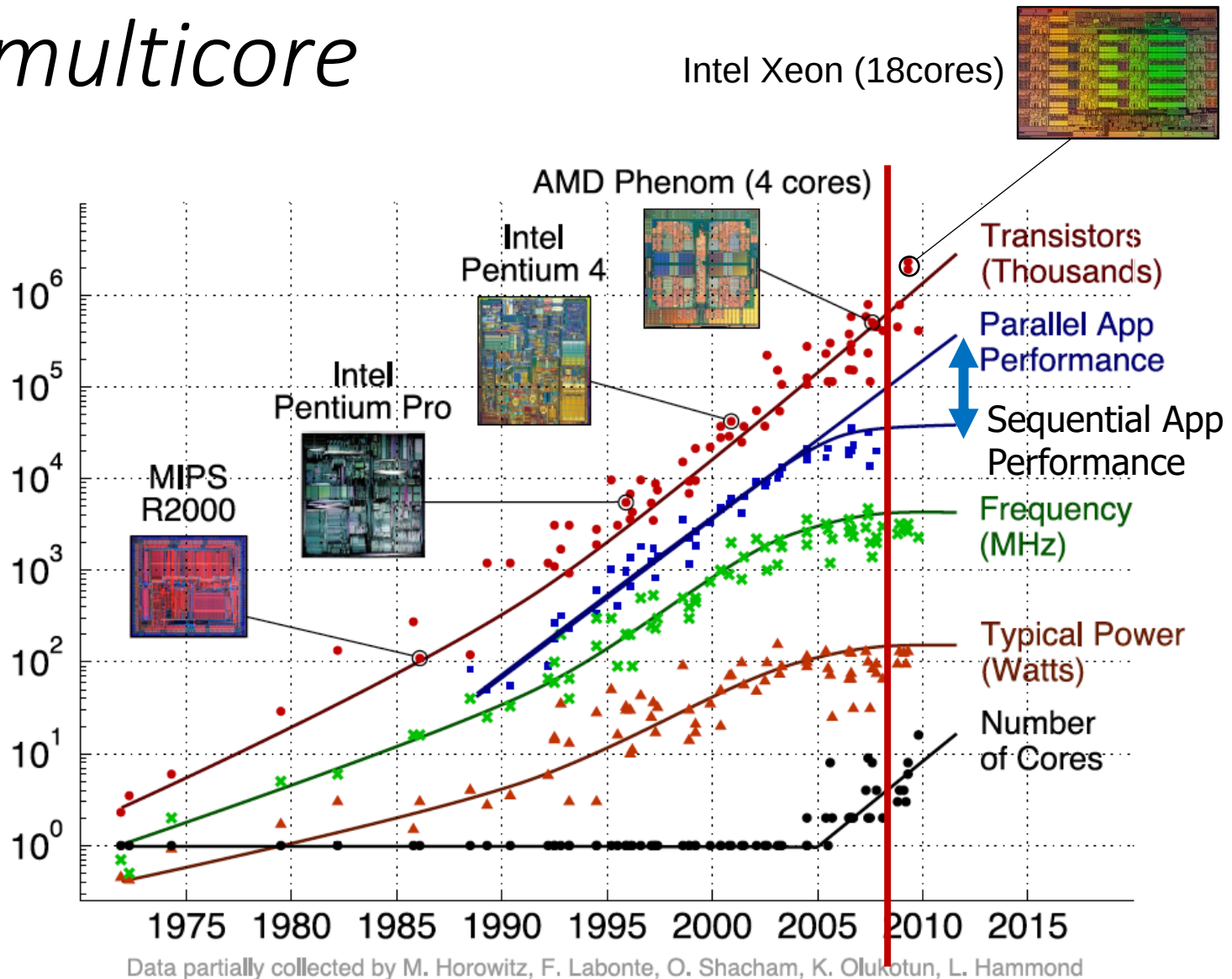
È vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

È inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia.

Credits

- Hennessy, Patterson, “Computer Organization and Design - RISC-V Edition : The Hardware Software Interface”, Morgan Kaufmann

I *multicore*



Tante forme di parallelismo

- Instruction level parallelism (ILP)
 - Pipeline
 - Multiple Issue
 - VLIW (superscalar)
 - Out-of-order schedulers
 - Speculation (Branch/Load predictors)
- Thread-level parallelism (TLP) (multicore)
 - Loop-level parallelism
 - Task-level parallelism
 - Memory-level parallelism
 - SIMD

Tante forme di parallelismo

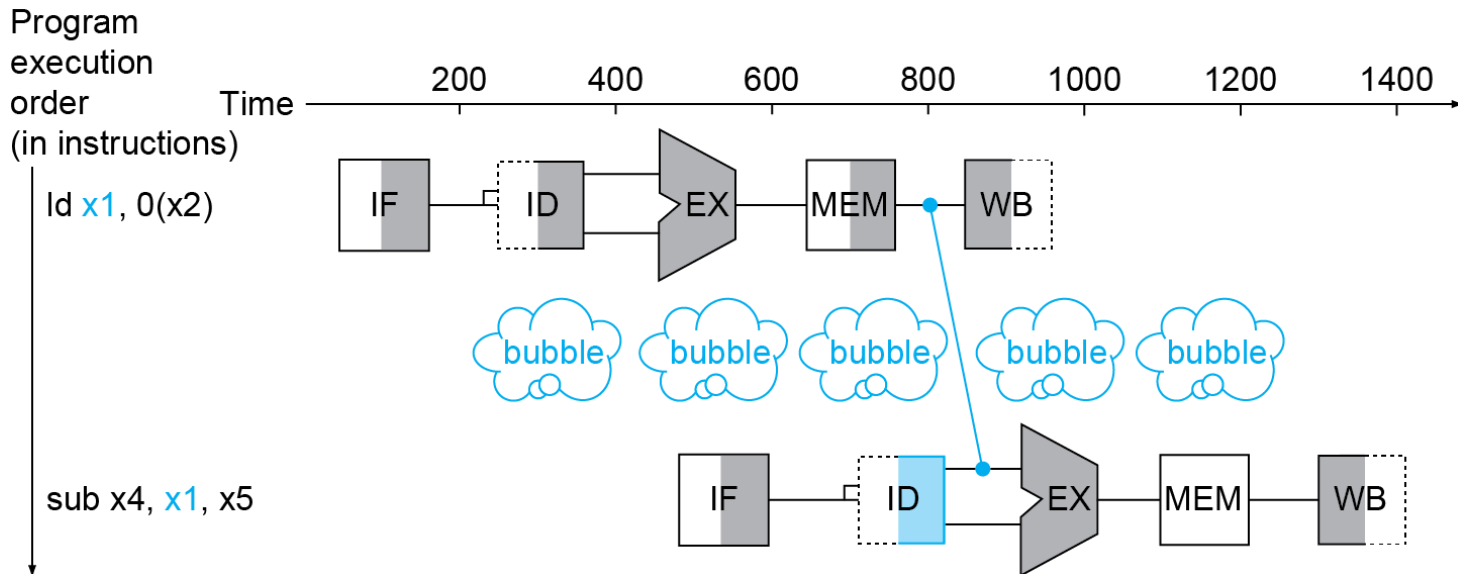
- Instruction level parallelism (ILP)
 - Pipeline
 - Multiple Issue
 - VLIW (superscalar)
 - Out-of-order schedulers
 - Speculation (Branch/Load predictors)
- Thread-level parallelism (TLP) (multicore)
 - Loop-level parallelism
 - Task-level parallelism
 - Memory-level parallelism
 - SIMD

ILP - Pipelining

- Il design *pipelined* di una CPU promette miglior performance
 - grazie allo sfruttamento del parallelismo a livello di stadi di esecuzione della CPU
 - Minor periodo di clock → maggiore frequenza
- Espone però la problematica degli *hazards* e degli **stalli**
 - **Code reordering** come strumento fondamentale per nascondere le latenze degli stalli
 - **Loop unrolling**

Load-Use Data Hazard

- Il forwarding aiuta a evitare gli stalli
 - Ma in certi casi non è sufficiente
 - Non si può propagare un valore indietro nel tempo!



Load-Use Data Hazard

- Il *code reordering* è la tecnica compiler-level più utilizzata per gestire il problema

```
ld      x5, 0(x10)
```

stall

```
sub      x6, x5, x11
```

```
slli     x7, x11, 2
```

```
add      x7, x7, x10
```



```
ld      x5, 0(x10)
```

```
slli     x7, x11, 2
```

```
sub      x6, x5, x11
```

```
add      x7, x7, x10
```

- Spesso presente all'interno dei loop
 - Il *loop unrolling* è preliminare al *code reordering*

Loop Unrolling

- Replica esplicitamente il loop body
 - Per esporre maggiore parallelismo
 - Per ridurre l'overhead di controllo del loop
 - ...
- Usa diversi registri per replica del loop body
 - register renaming
 - Evita loop-carried “anti-dependencies”
 - Una store seguita da una load sullo stesso registro
 - Non una dipendenza reale, ma piuttosto “di nome”
 - Ovvero, il riuso del nome di un registro

Scheduling Example

- Si consideri l'esecuzione di questo loop

```
Loop: ld    x31, 0(x20)      // x31=array element
      add   x31, x31, x21    // add scalar in x21
      sd    x31, 0(x20)     // store result
      addi  x20, x20, -8     // decrement pointer
      blt   x22, x20, Loop   // branch if x22 < x20
```

Loop Unrolling Example

- Loop srotolato di un fattore 2

Loop:	ld	x31, 0(x20)	// x31=array element
iteration	add	x31, x31, x21	// add scalar in x21
0	sd	x31, 0(x20)	// store result
	ld	x31, -8(x20)	// x31=array element
iteration	add	x31, x31, x21	// add scalar in x21
1	sd	x31, -8(x20)	// store result
	addi	x20, x20, -16	// decrement pointer
	blt	x22, x20, Loop	// branch if x22 < x20

Loop Unrolling Example

- Loop srotolato di un fattore 2

Devo rinominare i
registri per evitare
antidipendenze

Loop:	ld	x31, 0(x20)	// x31=array element
iteration	add	x31, x31, x21	// add scalar in x21
0	sd	x31, 0(x20)	// store result
iteration	ld	x31, -8(x20)	// x31=array element
1	add	x31, x31, x21	// add scalar in x21
	sd	x31, -8(x20)	// store result
	addi	x20, x20, -16	// decrement pointer
	blt	x22, x20, Loop	// branch if x22 < x20

Loop Unrolling Example

- Loop srotolato di un fattore 2

Loop:	ld	x30, 0(x20)	// x30=array element
iteration	add	x30, x30, x21	// add scalar in x21
0	sd	x30, 0(x20)	// store result
	ld	x31, -8(x20)	// x31=array element
iteration	add	x31, x31, x21	// add scalar in x21
1	sd	x31, -8(x20)	// store result
	addi	x20, x20, -16	// decrement pointer
	blt	x22, x20, Loop	// branch if x22 < x20

Loop Unrolling Example

- Loop srotolato di un fattore 2

**Si considerino ora
i cicli di stallo**

	Loop:	ld	x30, 0(x20)	// x30=array element
		stall		
iteration		add	x30, x30, x21	// add scalar in x21
0		sd	x30, 0(x20)	// store result
		ld	x31, -8(x20)	// x31=array element
		stall		
iteration		add	x31, x31, x21	// add scalar in x21
1		sd	x31, -8(x20)	// store result
		addi	x20, x20, -16	// decrement pointer
		blt	x22, x20, Loop	// branch if x22 < x20

Loop Unrolling Example

- Loop srotolato di un fattore 2

Code reordering per evitare gli stalli

```
Loop: ld    x30,0(x20)           // x30=array element
      ld    x31,-8(x20)         // x31=array element
      add   x30,x30,x21          // add scalar in x21
      add   x31,x31,x21          // add scalar in x21
      sd    x30,0(x20)           // store result
      sd    x31,-8(x20)         // store result

      addi   x20,x20,-16         // decrement pointer
      blt    x22,x20,Loop        // branch if x22 < x20
```

ILP – Multiple Issue

- Il concetto è semplice: aumentiamo il parallelismo replicando gli stadi della pipeline
 - O sfruttando multiple pipelines
- Possiamo così eseguire più istruzioni per ciclo di clock
 - Oltre quanto già consentito dal semplice pipelining
- Il CPI (clocks per instruction) diventa < 1
 - Il suo reciproco, l'Instructions Per Cycle (IPC) diventa > 1
- Es., 4GHz 4-way multiple-issue
 - peak CPI = 0.25, peak IPC = 4
 - Ma le dipendenze riducono i numeri...

ILP – Multiple Issue

- Static multiple issue
 - Occorre popolare gli “issue slots” con istruzioni che possono eseguire in parallelo
 - Il compilatore identifica gli *hazards* e ne evita gli effetti scegliendo l’ordine delle istruzioni in maniera opportuna (instruction scheduling)
- Dynamic multiple issue
 - La CPU osserva lo stream di istruzioni e seleziona quali di queste possono cominciare l’esecuzione ad ogni ciclo
 - Il compilatore può, in fase di *instruction scheduling*, aiutare l’esecuzione proponendo un ordine migliore
 - La CPU gestisce autonomamente gli *hazards* a runtime grazie ad hardware dedicato

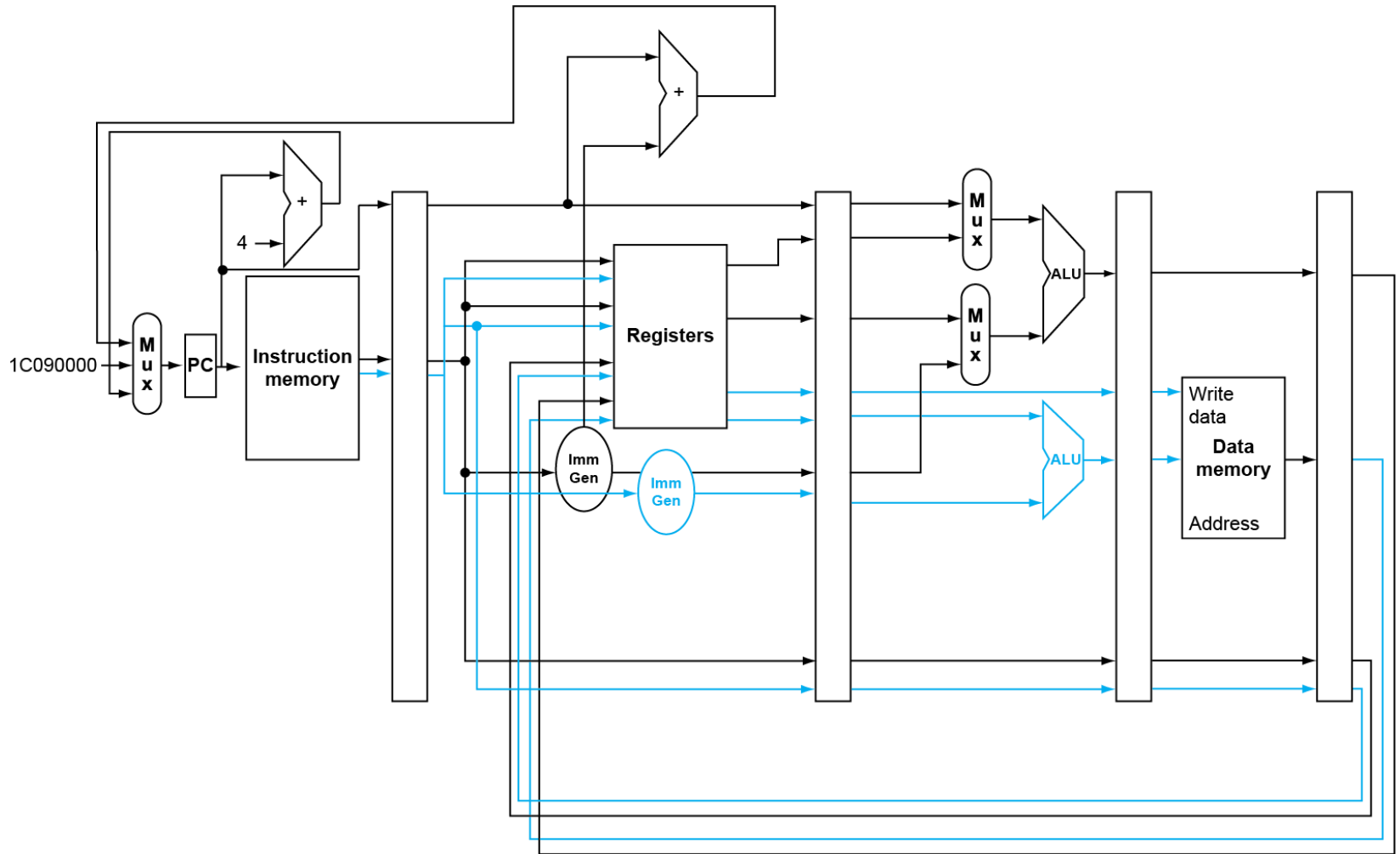
Static Multiple Issue

- Compiler groups instructions into “issue packets”
 - Group of instructions that can be issued on a single cycle
 - Determined by pipeline resources required
- Think of an issue packet as a very long instruction
 - Specifies multiple concurrent operations
 - $\Rightarrow$ Very Long Instruction Word (VLIW)

Scheduling Static Multiple Issue

- Compiler must remove some/all hazards
 - Reorder instructions into issue packets
 - No dependencies within a packet
 - Possibly some dependencies between packets
 - Varies between ISAs; compiler must know!
 - Pad with **nop** if necessary

RISC-V with Static Dual Issue



RISC-V with Static Dual Issue

- Two-issue packets
 - One ALU/branch instruction
 - One load/store instruction
 - 64-bit aligned
 - ALU/branch, then load/store
 - Pad an unused instruction with nop

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

Hazards in the Dual-Issue RISC-V

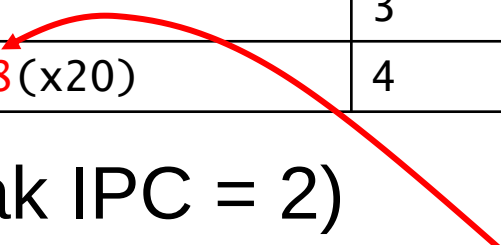
- More instructions executing in parallel
- EX data hazard
 - Forwarding avoided stalls with single-issue
 - Now can't use ALU result in load/store in same packet
 - add x10, x0, x1
 - ld x2, 0(x10)
 - Split into two packets, effectively a stall
- Load-use hazard
 - Still one cycle use latency, but now two instructions
- More aggressive scheduling required

Scheduling Example

- Schedule this for dual-issue RISC-V

```
Loop: ld    x31,0(x20)      // x31=array element
      add   x31,x31,x21     // add scalar in x21
      sd    x31,0(x20)     // store result
      addi  x20,x20,-8      // decrement pointer
      blt   x22,x20,Loop    // branch if x22 < x20
```

	ALU/branch	Load/store	cycle
Loop:	nop	ld x31,0(x20)	1
	addi x20,x20,-8	nop	2
	add x31,x31,x21	nop	3
	blt x22,x20,Loop	sd x31,8(x20)	4



■ $IPC = 5/4 = 1.25$ (c.f. peak $IPC = 2$)

need to add 8 because content of x20
was decreased in the meantime

Loop Unrolling con Dual Issue

Loop: iteration 0	ld	x31, 0(x20)	// x31=array element
	add	x31, x31, x21	// add scalar in x21
	sd	x31, 0(x20)	// store result
iteration 1	ld	x31, -8(x20)	// x31=array element
	add	x31, x31, x21	// add scalar in x21
	sd	x31, -8(x20)	// store result
iteration 2	ld	x31, -16(x20)	// x31=array element
	add	x31, x31, x21	// add scalar in x21
	sd	x31, -16(x20)	// store result
iteration 3	ld	x31, -24(x20)	// x31=array element
	add	x31, x31, x21	// add scalar in x21
	sd	x31, -24(x20)	// store result
addi x20, x20, -32			// decrement pointer
blt x22, x20, Loop			// branch if x22 < x20

Loop Unrolling con Dual Issue

Loop: iteration 0	ld	x28,0(x20)	// x31=array element
	add	x28,x28,x21	// add scalar in x21
	sd	x28,0(x20)	// store result
iteration 1	ld	x29,-8(x20)	// x31=array element
	add	x29,x29,x21	// add scalar in x21
	sd	x29,-8(x20)	// store result
iteration 2	ld	x30,-16(x20)	// x31=array element
	add	x30,x30,x21	// add scalar in x21
	sd	x30,-16(x20)	// store result
iteration 3	ld	x31,-24(x20)	// x31=array element
	add	x31,x31,x21	// add scalar in x21
	sd	x31,-24(x20)	// store result
	addi	x20,x20,-32	// decrement pointer
	blt	x22,x20,Loop	// branch if x22 < x20

Loop Unrolling Example

	ALU/branch	Load/store	cycle
Loop:	addi x20,x20,-32	ld x28, 0(x20)	1
	nop	ld x29, 24(x20)	2
	add x28,x28,x21	ld x30, 16(x20)	3
	add x29,x29,x21	ld x31, 8(x20)	4
	add x30,x30,x21	sd x28, 32(x20)	5
	add x31,x31,x21	sd x29, 24(x20)	6
	nop	sd x30, 16(x20)	7
	blt x22,x20,Loop	sd x31, 8(x20)	8

- $IPC = 14/8 = 1.75$
 - Closer to 2, but at cost of registers and code size

Dynamic Multiple Issue

- “Superscalar” processors
- CPU decides whether to issue 0, 1, 2, ... each cycle
 - Avoiding structural and data hazards
- Avoids the need for compiler scheduling
 - Though it may still help
 - Code semantics ensured by the CPU

Dynamic Pipeline Scheduling

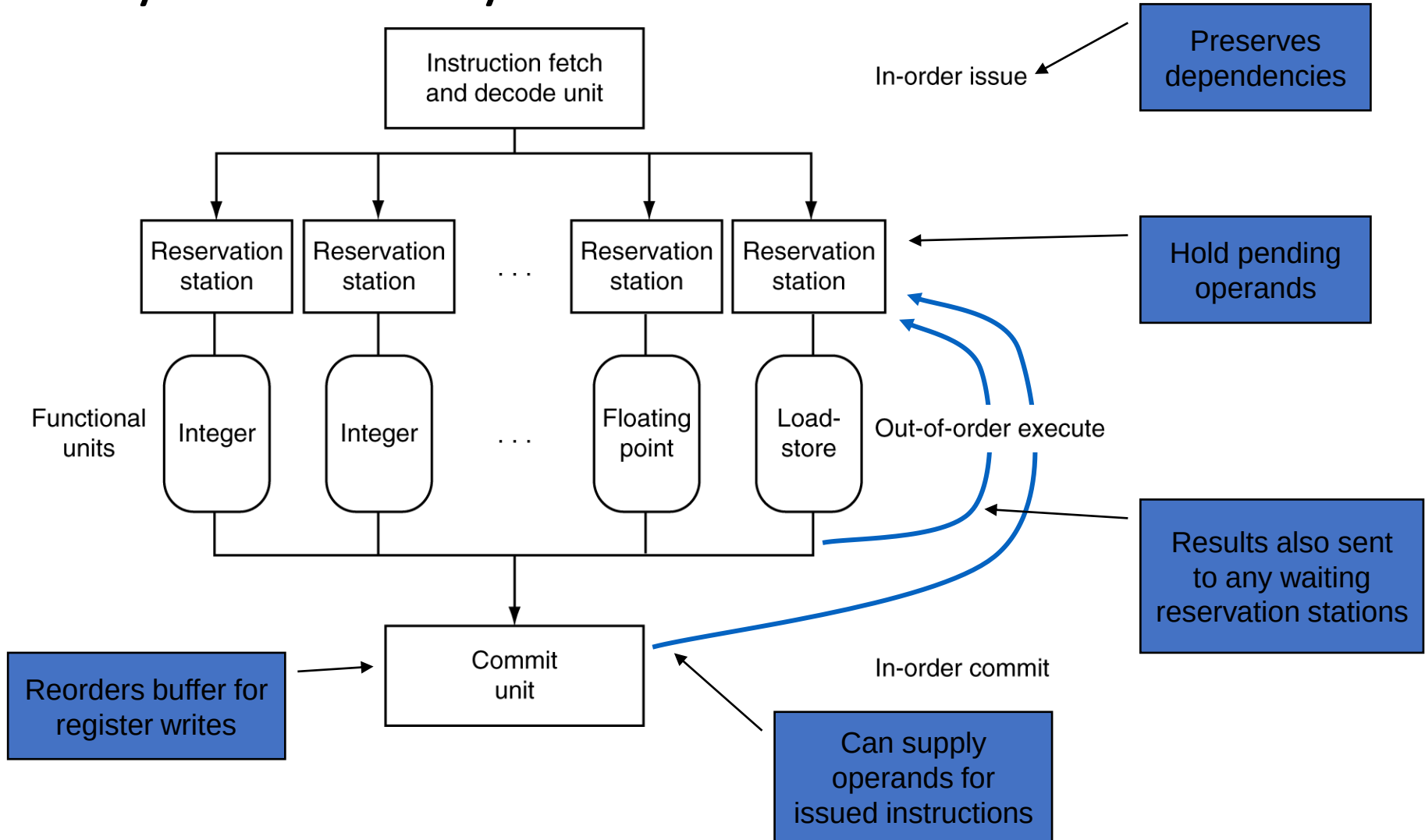
- Allow the CPU to execute instructions out of order to avoid stalls
 - But commit result to registers in order

- Example

```
ld    x31, 20(x21)
add   x1, x31, x2
sub   x23, x23, x3
andi  x5, x23, 20
```

- Can start sub while add is waiting for ld

Dynamically Scheduled CPU



Register Renaming

- Reservation stations and reorder buffer effectively provide register renaming
- On instruction issue to reservation station
 - If operand is available in register file or reorder buffer
 - Copied to reservation station
 - No longer required in the register; can be overwritten
 - If operand is not yet available
 - It will be provided to the reservation station by a function unit
 - Register update may not be required

Why Do Dynamic Scheduling?

- Why not just let the compiler schedule code?
- Not all stalls are predictable
 - e.g., cache misses
- Can't always schedule around branches
 - Branch outcome is dynamically determined
- Different implementations of an ISA have different latencies and hazards

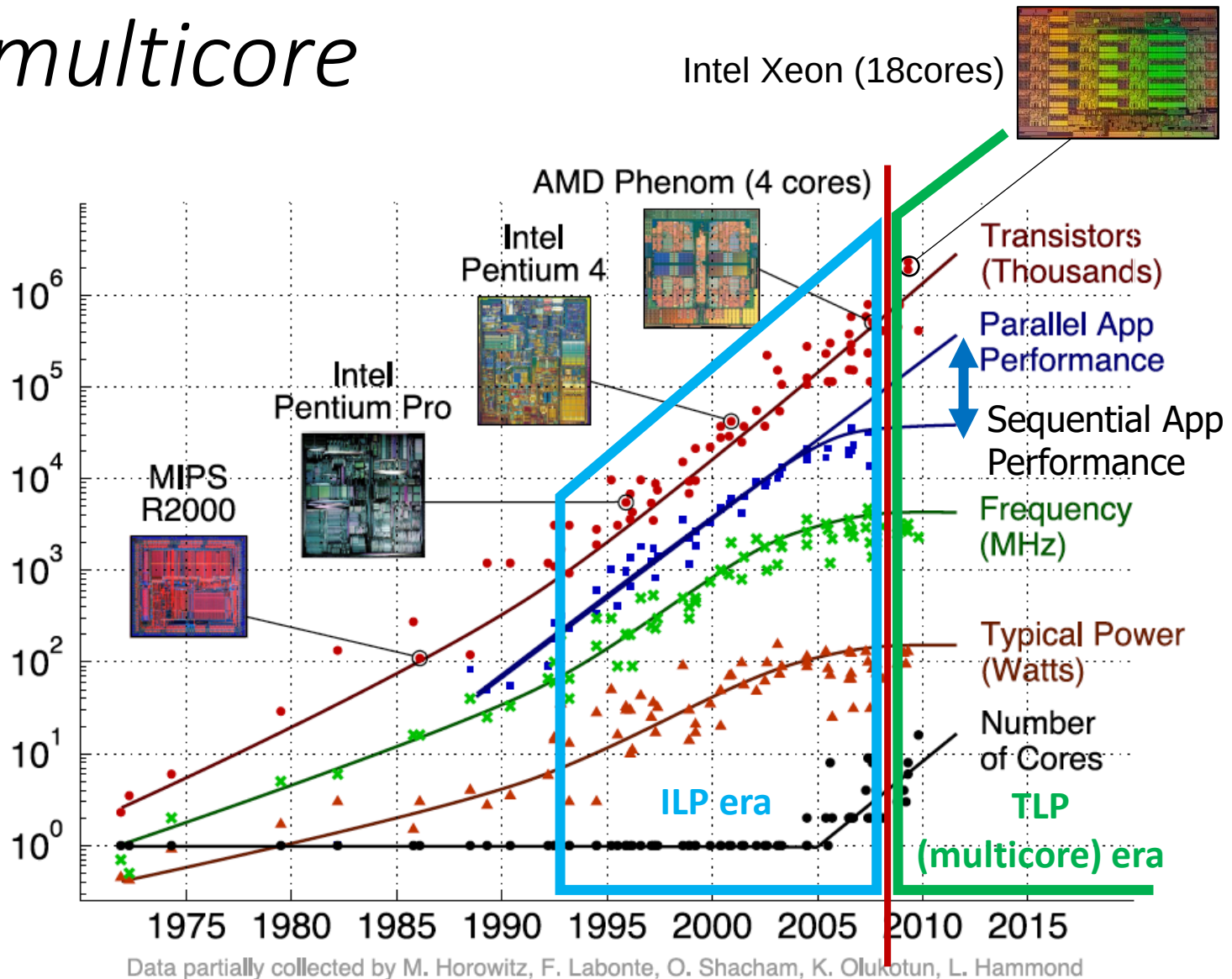
Does Multiple Issue Work?

- Yes, but not as much as we'd like
- Programs have real dependencies that limit ILP
- Some dependencies are hard to eliminate
 - e.g., pointer aliasing
- Some parallelism is hard to expose
 - Limited window size during instruction issue
- Memory delays and limited bandwidth
 - Hard to keep pipelines full
- Speculation can help if done well

Speculation

- Predict branch and continue issuing
 - Don't commit until branch outcome determined
- Load speculation
 - Avoid load and cache miss delay
 - Predict the effective address
 - Predict loaded value
 - Load before completing outstanding stores
 - Bypass stored values to load unit
 - Don't commit load until speculation cleared

I *multicore*



I *multicore*

- La perdita di performance dovuta al mancato incremento di frequenza è recuperata dal parallelismo (numero di *cores*)
- Potenzialmente la performance dei chip continua a crescere ad ogni generazione
- Non è però sfruttabile senza un cambio di paradigma nella maniera in cui si scrive il software
- I compilatori rivestono un ruolo fondamentale in questo contesto

Tante forme di parallelismo

- Instruction level parallelism (ILP)
 - Pipeline
 - Multiple Issue
 - VLIW (superscalar)
 - Out-of-order schedulers
 - Speculation (Branch/Load predictors)
- Thread-level parallelism (TLP) (multicore)
 - Loop-level parallelism
 - Task-level parallelism
 - Memory-level parallelism
 - SIMD

Thread-level Parallelism

- L'evoluzione dei sistemi di calcolo e dei sistemi operativi in ambienti multi-tasking offre una naturale maniera di sfruttare il parallelismo architetturale
 - Un processo per core
- Ma non è l'unica, o la più efficiente maniera di sfruttare il parallelismo hardware

Thread-level Parallelism

- Loop-level parallelism
- Probabilmente la più diffusa forma di parallelismo in ambienti scientifici
 - Oltre il 90% del tempo nelle applicazioni scientifiche è speso dentro i loop
 - Torna a presentarsi il problema delle dipendenze...

Loop-level Parallelism

- DOALL loops
 - Embarassingly parallel programs
 - Nessuna dipendenza

```
for (i=0; i<N; i++)  
    A[i] = B[i]
```

**Diverse iterazioni del loop
possono essere eseguite in
parallelo su diversi processori**

Loop-level Parallelism

- Loop-carried dependencies
 - Nei loop il problema delle dipendenze si manifesta tipicamente "attraverso" le iterazioni

```
for (i=1; i<N; i++)  
    A[i] = A[i-1];
```

Loop unrolled

```
A[1] = A[0];  
A[2] = A[1];  
...
```



Loop-level Parallelism

- Auto-parallelizing compilers
- A lungo si è cercato di creare ottimizzazioni nel compilatore che parallelizzassero automaticamente i loops
 - Dependence analysis
 - Analizza le espressioni di accesso alle strutture dati (funzioni delle induction variables) per scoprire se può verificarsi la situazione in cui in due iterazioni diverse puntano allo stesso elemento
 - Moltissimi test e frameworks (polyhedral analysis)
 - Code generation
 - Function outlining
 - Loop iteration space distribution

Loop-level Parallelism

- Auto-parallelizing compilers
- Di fatto tutti i maggiori compilatori moderni includono un passo di autoparallelizzazione
 - E autovettorizzazione (SIMD)
 - Funziona bene per loop regolari
- Di fatto esistono molte situazioni nelle quali anche le analisi più sofisticate non sono in grado di estrarre il parallelismo
 - Il programmatore diventa "responsabile" per l'esposizione del parallelismo
 - Parallel programming models